RIS COURSE PROJECT · FINAL PRESENTATION

# QPASE, built and improved

Implementing quantum-resistant password-authenticated searchable encryption, and making its search forward-private

Based on Jiang & Wang · IEEE Transactions on Information Forensics and Security, 2024


PART 1

**The paper and our goal**

Shambo Roy · 2026801007

PART 2

**Implementing QPASE**

Shambhavi Gantla · 2026801014

PART 3

**The counter-based index**

Iman Kalyan Chakraborty · 2026801003

PART 1 · THE PAPER

# From keys to passwords to post-quantum

SSE · since 2000

### Key-based search

The server searches ciphertext, but the user needs a long random key.

Key must live on each device

PASE · Chen et al. 2021

### Password-based search

User holds only a password. Servers help rebuild the key.

Discrete-log: broken by Shor

QPASE · Jiang & Wang 2024

### Quantum-safe password search

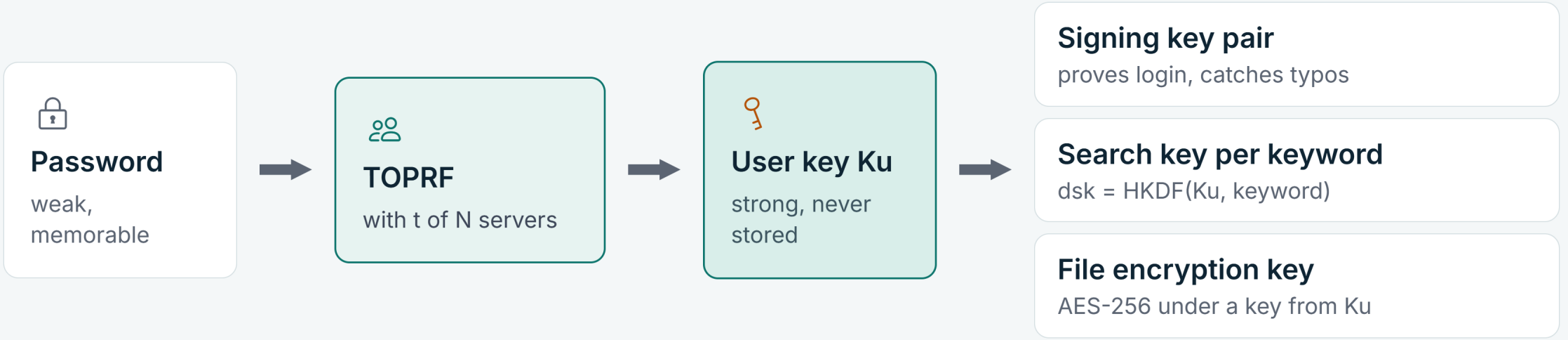
Same password-only promise, rebuilt on lattices.

**No known quantum attack on LWE**

QPASE is the first password-authenticated searchable encryption designed to resist a quantum adversary.
**This is the scheme we implemented.**

PART 1 · THE PAPER

# How QPASE works




1 · REGISTER
derive Ku, give servers pk

2 · LOGIN
rebuild Ku, prove it

3 · OUTSOURCE
encrypt file and tag, upload

4 · RETRIEVE
send dsk, get matches

+ KEY UPDATE
servers refresh shares

# The three blocks that carry the scheme

HARDNESS

## Decision-LWE

Noisy linear equations look random. Removing the noise is the hard problem.

**Why: no known quantum attack**

REGISTER AND LOGIN

## Lattice TOPRF

The user blinds the password; t servers apply key shares without seeing it.

**Why: strong key, no offline guessing**

ON THE TOPRF OUTPUT

## Robust extractor

A 1-bit hint plus rounding removes the lattice noise from the output.

**Why: same Ku at every login**

**Also used:** HKDF + PRF · per-keyword keys | ML-DSA-44 signatures | DKG · no trusted dealer | AES-256-GCM

PART 1 · OUR GOAL

# What QPASE leaves open in search

PROBLEM 1

### The search key is permanent

To search, the user hands the server dsk. The server can keep it and test every later upload.

**No forward privacy**

PROBLEM 2

### Every search is a full scan

Each file carries a locked tag. The server tries dsk on every stored tag, one by one.

**Every tag is tried, and each match costs the server a lattice PRF**

WHAT WE SET OUT TO DO

**1 Reproduce**
QPASE end to end

→

**2 Improve**
a counter-based index

→

**3 Measure**
both, at N = 3, 5, 7

# How our implementation is built

**Notebooks**
notebooks/

- 01 building blocks
- 02 register, login
- 03 outsource, retrieve
- 04 key update
- 05 costs
- 06 PARAM II test
- 07 counter index
- 08 search scaling

**Protocol**
protocol.py

- runs whole phases
- records every message and its exact size, via display.py

**Parties**
parties/

- QPASEUser
- QPASEServer
- CounterIndexUser / Server, extending them

**Primitives**
primitives/

- ring Rq
- lattice PRF
- robust extractor
- Shamir + DKG
- TOPRF
- HKDF + PRF
- ML-DSA-44
- AES-256-GCM
- linear hash

**Python 3.14** with numpy, cryptography and dilithium-py, managed by uv

**Every number we report** is printed by a named notebook cell and checked by a script

PART 2 · OUR IMPLEMENTATION

# Where the paper left us choices

## Ring or plain LWE?

Paper: Fig. 1 uses plain-LWE matrices, the scheme text a ring.

**Ours:** the ring Zq[X]/(X^n + 1). Plain LWE is infeasible in Python.

## Extractor hint

Paper: Fig. 2 takes the hint from an unrelated value.

**Ours:** hint made at Register, stored by servers, returned at Login.

## TOPRF combine

Paper: the proof prints one sign two different ways.

**Ours:** the sign the algebra gives; only server sets with integer Lagrange coefficients.

## Values left open

Paper: p, σ, AES mode and Dilithium level unspecified.

**Ours:** p = 2, σ = 3.2, AES-256-GCM, ML-DSA-44.

L = 128 is given in the paper.

# Our QPASE works end to end

LOGIN

**10 / 10**

logins gave a byte-identical Ku

notebook 02, cell 10

RATE LIMIT

**μ = 14**

attempts, then the server refuses

notebook 02, cell 11

SEARCH

**1, 3, 6**

items found for “medical”, as stored

notebook 03, cell 12

KEY UPDATE

**9 / 9**

shares accepted; users log in as before

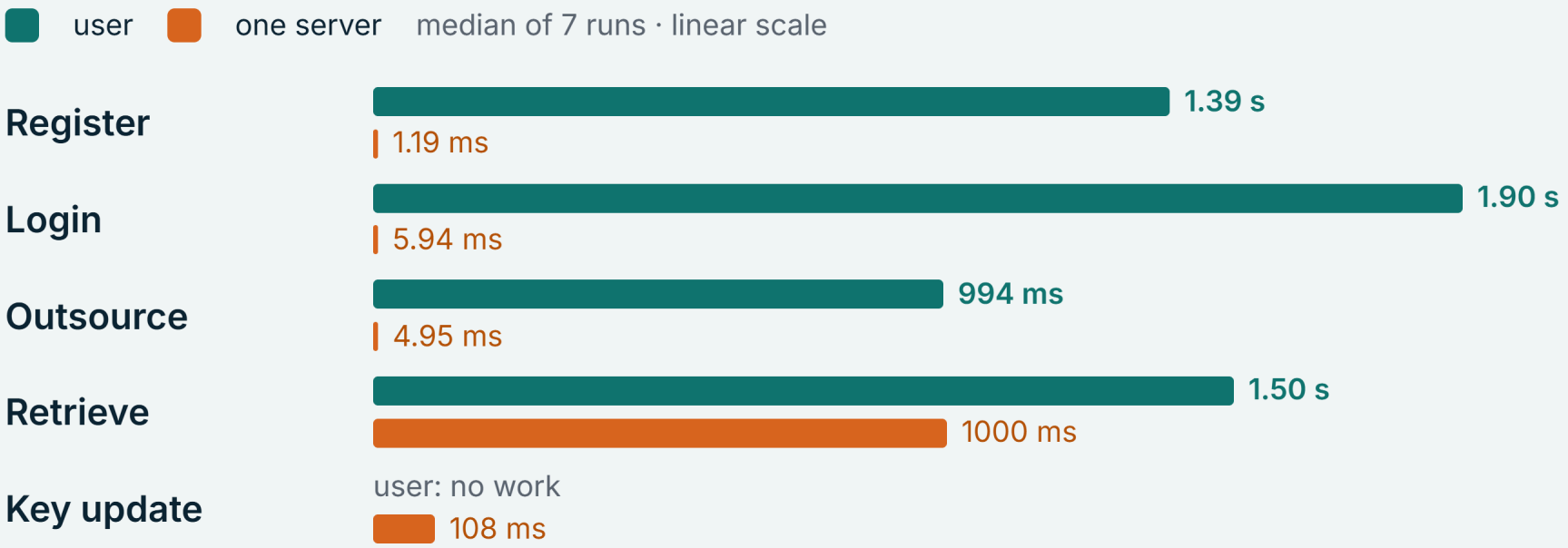
notebook 04, cells 6 and 11

**184** unit tests pass

**0** failed checks in 270 timed runs

PART 2 · MEASURED COST

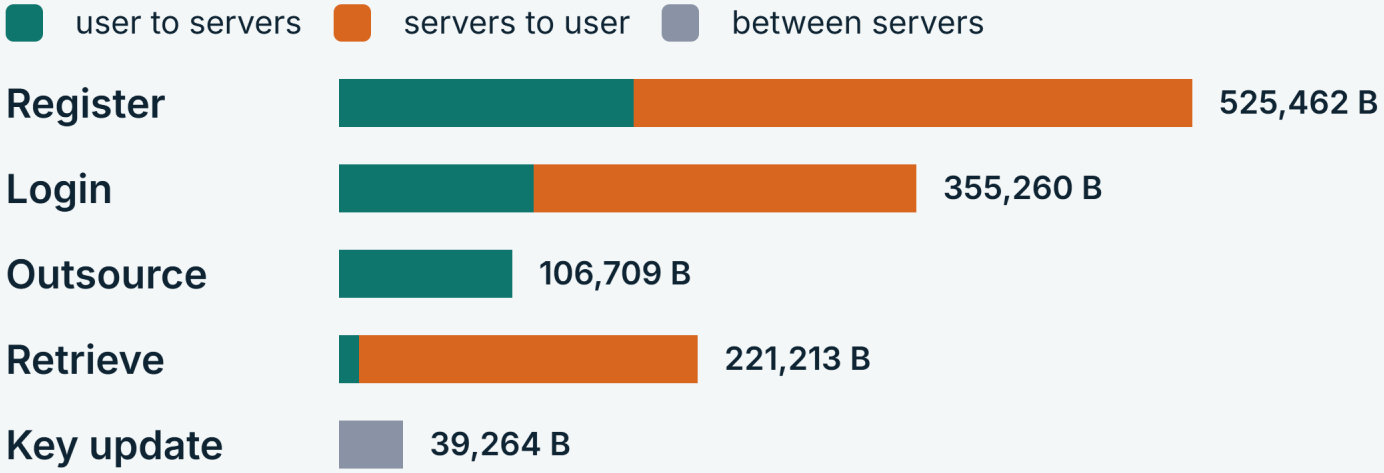
# Computation time per phase




**The lattice PRF is the bottleneck.** One evaluation takes about 492 ms in Python. Together, the evaluations are 92–102% of the user's time in every phase.

PART 2 · MEASURED COST

# Bytes on the wire per phase




Outsource: one 100 KB file · Retrieve: two matching 100 KB files

AGAINST THE PAPER

Same order as the paper's Table IV: Register ≈ 550 KB and Login ≈ 262 KB, with 2 servers.

Login grows with the servers contacted: each L1 reply is 117,792 B.

Confirms the paper's own caveat: heavy on bandwidth.

PART 2 · CHECKPOINT 1 OF 3

# Reproduced

Our Python QPASE runs every phase of the paper: the same key at every login, correct search results, and users unaffected by key updates.

**5 / 5**
phases built, key update included
notebooks 02 to 04

**184**
unit tests pass
unittest suite

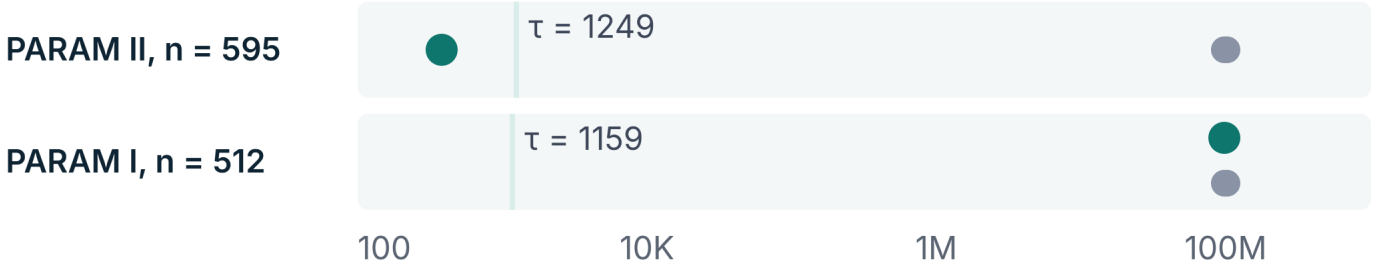
**492 ms**
per lattice PRF evaluation, the main cost
notebook 05, cell 3

**355 KB**
per login with 2 server contacts: heavy, as the paper warns
notebook 05, cell 11

# Two security gaps we found

GAP 1 · OFFLINE GUESSING AT PARAM II

n = 595 is odd, so X = −1 is a root of X^595 + 1. One server can test password guesses offline against one blinded login.




**Teal dot:** real password's residual. **Grey:** wrong guesses. **Line:** match threshold τ. Log scale. At PARAM II the real password scores 382 and wrong guesses start at 88,243,150, so "sunshine1" is found. At PARAM I, nothing matches.

GAP 2 · KEY UPDATE

## Commitments hide nothing

Sec. IV-F needs a linear hash H. Ours, H(x) = A·x mod q, can be inverted, so an eavesdropper learns every update share.

**Fix needs a hiding commitment**

PART 2 · CHECKPOINT 2 OF 3

# Found

Building the scheme exposed what reading the paper did not: two security gaps, and a specification with holes.

SECURITY GAP

### Offline guessing at PARAM II

In the ring reading of n = 595, one server finds the password offline.

real password 382, threshold τ 1249

SECURITY GAP

### Key-update commitments

The linear hash can be inverted, so every update share is visible.

needs a hiding commitment

SPECIFICATION

### Holes in the paper

Ring or plain LWE, the extractor hint, the TOPRF sign and open values each needed a decision.

all documented in the report

# Our counter-based search index

WHAT ONLY THE USER CAN WORK OUT

| Keyword | j | Label = PRF(Kw, j) | File |
|---|---|---|---|
| invoices | 1 | 3b90…5e | A |
| invoices | 2 | a47d…09 | C |
| invoices | 3 | d6f1…72 | D |
| contracts | 1 | 1f7c…a3 | B |
| contracts | 2 | 8e12…c4 | E |

WHAT THE SERVER STORES

| Label | Entry |
|---|---|
| 1f7c…a3 | sealed pointer to file B |
| 3b90…5e | sealed pointer to file A |
| 8e12…c4 | sealed pointer to file E |
| a47d…09 | sealed pointer to file C |
| d6f1…72 | sealed pointer to file D |

### Key stays on the device

Kw = HKDF(Ku, w) is the old dsk. It is never sent.

### Counter table

invoices = 3, contracts = 2. Sealed and signed; a copy on every server.

### Small rows

32 B label + 108 B entry per file. The paper’s tag Ct is 4,256 B.

# The phases that change

## Login

| | |
|---|---|
| L1–3 | as in QPASE: rebuild Ku, prove it |
| L4 | **new:** fetch the sealed counter table; keep the newest that verifies |

## Outsource

| | |
|---|---|
| O1 | send the row for j = c_w + 1 and the file C to one server |
| O2 | set c_w = j; upload the new counter table to all N servers |

## Retrieve

| | |
|---|---|
| R1 | labels 1 to c_w to all N |
| R2 | c_w lookups, **no scan** |
| R3 | user opens the entries |
| R4 | fetch files by item id |
| R5 | check hash, then decrypt |

**Unchanged:** Register, Login L1–L3 and the server key update. Ku survives a key update, so every label stays valid.

PART 3 · RESULTS

# Forward privacy, shown in code

STEP 1

### Search “medical”

3 labels sent. Items 1, 3, 6 found with 3 lookups per server. A server keeps the request.

STEP 2

### Upload item 8

Filed under “medical” with label 4. The server never had Kw, so it cannot compute that label.

STEP 3

### Server replays it

It sends the kept search request again, hoping to find the new upload.

OUR COUNTER INDEX

## Items 1, 3, 6

The replay misses item 8. A new search by the user finds 1, 3, 6, 8.

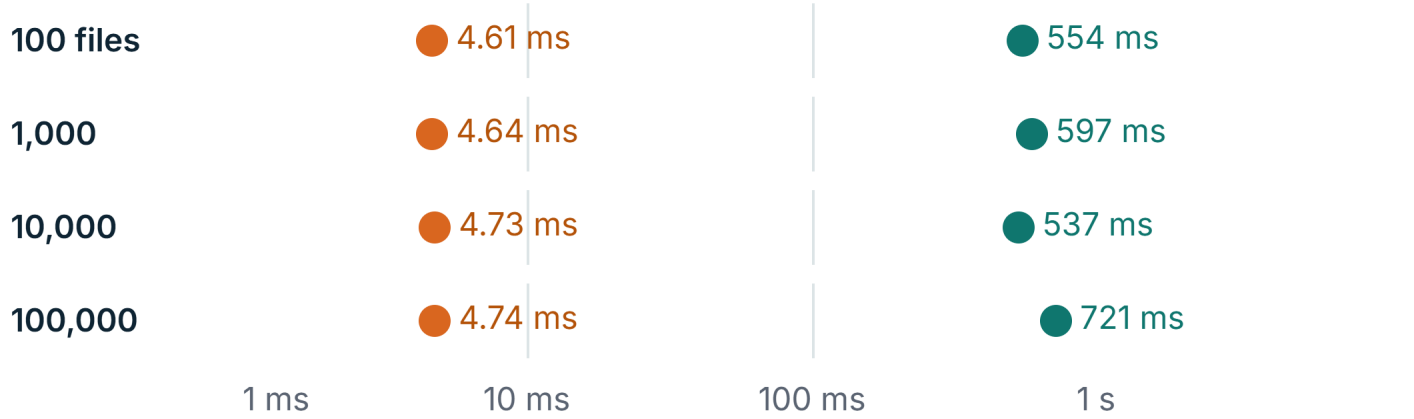
THE PAPER’S QPASE

## MATCH

A kept dsk, tested on a later upload, finds it at once.

PART 3 · RESULTS

# Search time against storage size




SPEED-UP, SLOWEST SERVER

| N | 100 files | 100,000 |
|---|---|---|
| 3 | 120× | 152× |
| 5 | 110× | 146× |
| 7 | 111× | 130× |

3 lookups per server in every run. The user's time stays about 2 s in both schemes.

PART 3 · RESULTS

# What the index still leaks and costs

## What a server learns

| Event | Server learns |
|---|---|
| Outsource | one random-looking row and the file |
| Search | c_w labels and which ones hit |
| Same search again | the same labels: search pattern |
| Upload after a search | nothing that links it: forward privacy |

## Costs we accept

- Every upload sends the counter table to all N servers: 3,497 B each
- Search request grows with matches: 2,521 B for c_w = 3
- Retrieve takes two round trips
- No deletion yet, so no backward privacy
- Two devices at once must sync counters

Same lattice core, same login: the index changes only how files are found.

# Improved

The counter index keeps QPASE’s lattice core and login, and changes only how files are found.

**Yes**

forward privacy: a kept request misses later uploads

notebook 07, cell 14

**109–152×**

faster search at the slowest server, N = 3, 5, 7

notebook 08, cell 7

**< 5 ms**

per server search, up to 100,000 stored files

notebook 08, cell 7

**140 B**

index row per file, against a 4,256 B tag

notebooks 07 and 03, cell 4

SUMMARY

# We built QPASE, found where it is weak, and made its search forward-private and faster.

REPRODUCED

Every phase in Python, key update included, with exact costs per phase.

FOUND

Offline guessing at PARAM II in the ring reading; non-hiding key-update commitments.

IMPROVED

Counter index: forward privacy shown; 109× to 152× faster at the slowest server.

Next: a hiding commitment for key update · plain-LWE PARAM II · deletion with backward privacy · tests over a real network

## Questions?